DESAFIO TÉCNICO: IA GENERATIVA

A Panvel é uma rede de farmácias com mais de 50 anos de história, e que atualmente conta com mais de 650 filiais espalhadas pelos estados do Rio Grande do Sul, Santa Catarina, Paraná e São Paulo. Além disso, a empresa investe fortemente em sua plataforma digital, oferecendo aos clientes a conveniência de comprar produtos online com a opção de entrega em domicílio, ou retirada em loja. Nossa missão é proporcionar saúde e bem-estar para as pessoas, e nossa visão é ser a melhor em produtos e serviços de saúde e bem-estar de forma sustentável e inovadora!

Desafio: Assistente Conversacional para Atendimento Farmacêutico

Todos os dias, milhares de clientes e times internos da Panvel buscam informação sobre medicamentos (indicações, posologia, contraindicações, interações), orientação sobre as filiais (qual loja mais próxima oferece Panvel Clinic, atende 24 horas, faz delivery) e respostas analíticas sobre o desempenho da operação (vendas por loja, atingimento de metas, comparativos por perfil de filial). Hoje boa parte dessa demanda é atendida por pessoas em diversos canais — site, WhatsApp, telefone, loja, BI manual — com alto custo e tempo de resposta variável.

Sua missão é projetar e implementar um **assistente conversacional baseado em LLM** que cubra duas classes de pergunta com qualidade, rastreabilidade e segurança:

1. **Informação farmacológica**, respondendo com base em bulas reais via **retrieval-augmented generation (RAG)**.
2. **Consulta a dados estruturados da rede de filiais**, via **chamadas a ferramentas (tool calling)**.

O foco deste case são as operações da Panvel no estado do Paraná (PR).

Objetivos e Entregáveis

Backend (API conversacional): Implemente um serviço que exponha um endpoint de chat com streaming de resposta (SSE ou equivalente), orquestre o ciclo de geração com tool calling, faça RAG sobre o corpus de bulas e mantenha contexto conversacional ao longo dos turnos.

Frontend (Interface de chat): Construa uma interface de chat que consuma a API, renderize a resposta em streaming token a token, mostre as fontes citadas (bulas referenciadas) e dê visibilidade ao usuário sobre quais ferramentas foram chamadas e com que argumentos.

Pipeline RAG: Faça a ingestão do corpus de bulas (chunking, geração de embeddings, indexação em um vector store de sua escolha — local ou em cloud) e a recuperação durante o chat, com citação explícita das fontes na resposta final. As bulas são documentos reais da Anvisa e variam em estrutura, extensão e qualidade de extração.

Tool Calling sobre filiais: Modele e implemente ao menos duas ferramentas que o LLM possa invocar para responder perguntas sobre filiais. Por exemplo: `buscar_filiais(cidade, servicos)` para listar filiais que atendem certos critérios, e `detalhes_filial(codigo_filial)` para retornar o cadastro completo de uma filial. Use `filiais.parquet` como fonte de verdade.

Observabilidade: Para cada turno do diálogo, registre dados que permitam auditoria e diagnóstico em produção — trace id, tokens consumidos, latência por etapa, ferramentas chamadas com seus argumentos, documentos recuperados e fontes citadas.

Critérios de Avaliação

- Qualidade da orquestração LLM (prompts, controle do loop de tool calling, prevenção de loops infinitos, tratamento de falhas).
- Profundidade e justificativa do pipeline RAG (estratégia de chunking adequada a documentos PDF heterogêneos, escolha do modelo de embedding, recuperação, citação).
- Modelagem das ferramentas (contratos claros, validação de entrada, mensagens de erro acionáveis para o LLM).
- Qualidade do frontend (UX de streaming, estados de carregamento e erro, exibição de fontes e tool calls).
- Arquitetura backend (separação de responsabilidades, testabilidade, possibilidade de trocar o provider LLM sem reescrever o domínio).
- Observabilidade e operabilidade da solução.
- Capacidade de comunicar decisões técnicas e trade-offs (README, ADRs ou equivalente).
- Recomendações estratégicas baseadas no que foi observado durante a construção.

Stack

Você tem **liberdade total** na escolha das tecnologias. Sugestões alinhadas ao stack utilizado pela Panvel: backend em Java com Spring Boot, ou em Python com FastAPI; frontend em Angular ou React. Qualquer provider LLM (OpenAI, Anthropic, AWS Bedrock, modelos locais via Ollama ou similares), qualquer framework de orquestração de LLM (Spring AI, LangChain, LangGraph, LlamaIndex, OpenAI SDK puro, ou nenhum) e qualquer vector store (Qdrant, pgvector, Chroma, Elasticsearch, FAISS, etc.) são aceitos. O critério é justificar a escolha — não usar o que está na moda.

Bases de Dados Disponíveis

As bases acompanham o desafio:

filiais.parquet: Cadastro das filiais Panvel no estado do Paraná, com atributos como cidade, faixa de vida (tempo de operação), tipo de estabelecimento, indicador de delivery, metragem da área de venda, presença de Panvel Clinic, estacionamento e atendimento 24 horas. Fonte de verdade para as ferramentas de busca de filiais.

corpus_bulas/: Conjunto de 20 bulas reais da Anvisa em PDF. Cobre diferentes classes terapêuticas (psicotrópicos controlados como metilfenidato e flunitrazepam, antidepressivos, anti-hipertensivos, antibióticos, AINEs, IBPs, contraceptivos, antialérgicos, entre outros). Estrutura, layout e qualidade de extração variam entre bulas — espelha o que ocorre em produção.

dicionario_dados.xlsx: Descrição dos campos da base de filiais e da lista de bulas disponíveis.

> *Observação: As bulas são documentos públicos da Anvisa e foram disponibilizadas exclusivamente para fins deste desafio. As respostas geradas pelo assistente devem sempre citar a bula de origem.*

Entrega

Repositório Git (público ou privado com acesso compartilhado) contendo o código backend, o código frontend, documentação de setup local (idealmente um único `docker compose up` *ou equivalente) e o documento de decisões arquiteturais. Após a entrega, faremos uma conversa com você para que apresente a arquitetura e justifique as escolhas técnicas.*